

SCHULPROJEKT — INFORMATIKER EFZ, 2. LEHRJAHR



# Adressverwaltung Änderungsprotokoll v2.2.0

*JWT-Authentifizierung im Backend*

September 2026 • Informatiker EFZ, 2. Lehrjahr

Next.js 14

C# ASP.NET Core 8

OData

PostgreSQL 16

JWT Bearer

## 1

## Übersicht der Änderungen

Version 2.2.0 führt **vollständige JWT-Authentifizierung (JSON Web Token)** im C#-Backend ein. Alle OData- und REST-Endpunkte sind nun durch einen Bearer-Token gesichert. Das Frontend speichert den Token in der NextAuth-Session und sendet ihn bei jedem API-Aufruf automatisch mit.

### Motivation: Warum JWT im Backend?

In Version 2.1.0 war lediglich das Next.js-Frontend durch NextAuth geschützt – das Backend selbst prüfte keinerlei Tokens. Eine direkte HTTP-Anfrage an `http://localhost:5000/odata/Adressen` lieferte Daten ohne Authentifizierung. Ab v2.2.0 ist die API von aussen nicht mehr ohne gültige Anmeldung abrufbar.

## Umfang der Änderungen

| Bereich                            | Anzahl Dateien                  | Art der Änderung                                 |
|------------------------------------|---------------------------------|--------------------------------------------------|
| Backend (C# ASP.NET Core 8)        | 7 Dateien                       | JWT registrieren, ausstellen, Endpunkte schützen |
| Frontend (Next.js 14 / TypeScript) | 3 Dateien                       | Token speichern, automatisch mitsenden           |
| Infrastruktur (Docker)             | <code>docker-compose.yml</code> | Umgebungsvariable <code>Jwt__Key</code> ergänzen |

### Kein Datenbank-Schema-Update nötig

Der JWT wird ausschliesslich im Arbeitsspeicher generiert und im verschlüsselten Session-Cookie des Browsers gespeichert. Es sind keine EF-Core-Migrationen erforderlich.

## 2

## Betroffene Dateien

### Backend (C# ASP.NET Core 8)

| Datei                                   | Art      | Beschreibung                                                                                                                   |
|-----------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>AdressverwaltungApi.csproj</code> | Geändert | NuGet-Paket <code>Microsoft.AspNetCore.Authentication.JwtBearer</code> v8.0.0 hinzugefügt                                      |
| <code>appsettings.json</code>           | Geändert | Neue Sektion <code>Jwt</code> mit <code>Key</code> , <code>Issuer</code> , <code>Audience</code> , <code>ExpiresInHours</code> |

|                                    |          |                                                                                                                                             |
|------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Program.cs                         | Geändert | JWT-Authentication registriert<br>( AddAuthentication , AddJwtBearer );<br>app.UseAuthentication() in Middleware-Pipeline eingefügt         |
| Dtos/AuthDtos.cs                   | Geändert | Neues Record LoginResponse mit Feld Token                                                                                                   |
| Controllers/AuthController.cs      | Geändert | Neue Methode GenerateJwt(User) : erstellt und signiert JWT; IConfiguration injiziert; Login gibt LoginResponse statt anonymes Objekt zurück |
| Controllers/ODataCrudController.cs | Geändert | [Authorize] -Attribut auf Basisklasse gesetzt → schützt AdressenController und CitiesController automatisch                                 |
| Controllers/SettingsController.cs  | Geändert | [Authorize] -Attribut hinzugefügt                                                                                                           |

## Frontend (Next.js 14 / TypeScript)

| Datei                | Art      | Beschreibung                                                                                                                                   |
|----------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| types/next-auth.d.ts | Geändert | Module Augmentation: User.accessToken , Session.accessToken , JWT.accessToken ergänzt                                                          |
| lib/auth.ts          | Geändert | authorize() : liest token aus Backend-Response; jwt -Callback: speichert accessToken ; session - Callback: gibt accessToken an Frontend weiter |
| lib/api.ts           | Geändert | Neue Funktion apiFetch() : liest Token via getSession() , setzt Authorization: Bearer ... - Header; alle 13 fetch() -Aufrufe ersetzt           |

### 3

## Detailbeschreibung der Änderungen

### 3.1 JWT-Konfiguration (appsettings.json)

Alle JWT-Parameter werden zentral in appsettings.json konfiguriert. Der Schlüssel muss in Produktivumgebungen über eine Umgebungsvariable gesetzt werden (niemals in Git einchecken).

appsettings.json (neuer Abschnitt)

```
"Jwt": {
  "Key":      "dein-geheimer-jwt-schlüssel-mindestens-32-zeichen-lang!",
  "Issuer":   "AdressverwaltungApi",
  "Audience": "AdressverwaltungFrontend",
  "ExpiresInHours": 8
}
```

### Docker: Schlüssel als Umgebungsvariable setzen

In `docker-compose.yml` den echten Schlüssel über die Umgebungsvariable übergeben – nie den Platzhalter aus `appsettings.json` verwenden:

`docker-compose.yml` (Umgebungsvariable)

```
environment:
  - Jwt__Key=dein-echter-sicherer-schlüssel-mindestens-32-zeichen
```

## 3.2 JWT-Authentifizierung registrieren (Program.cs)

In `Program.cs` wird der JWT-Bearer-Dienst registriert und in der Middleware-Pipeline aktiviert. Die Reihenfolge von `UseAuthentication()` vor `UseAuthorization()` ist zwingend.

`Program.cs` – JWT-Dienst registrieren

```
var jwtKey = builder.Configuration["Jwt:Key"]
?? throw new InvalidOperationException("Jwt:Key fehlt in appsettings.json");

builder.Services
  .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
  .AddJwtBearer(options =>
  {
    options.TokenValidationParameters = new TokenValidationParameters
    {
      ValidateIssuer      = true,
      ValidateAudience    = true,
      ValidateLifetime     = true,
      ValidateIssuerSigningKey = true,
      ValidIssuer          = builder.Configuration["Jwt:Issuer"],
      ValidAudience       = builder.Configuration["Jwt:Audience"],
      IssuerSigningKey     = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(jwtKey)),
    };
  });
```

`Program.cs` – Middleware-Reihenfolge (wichtig!)

```
app.UseAuthentication(); // ← muss VOR UseAuthorization stehen
app.UseAuthorization();
```

### 3.3 JWT ausstellen (AuthController)

Die private Methode `GenerateJwt(User)` erstellt und signiert den Token. Sie liest alle Parameter aus der Konfiguration – keine Magic Strings im Code.

Controllers/AuthController.cs – GenerateJwt

```
private string GenerateJwt(User user)
{
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var hours = double.Parse(_config["Jwt:ExpiresInHours"] ?? "8");

    var claims = new[]
    {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new Claim(JwtRegisteredClaimNames.Email, user.Email),
        new Claim(JwtRegisteredClaimNames.Name, user.DisplayName),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
    };

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddHours(hours),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Die Login-Methode gibt nun ein `LoginResponse` -Record zurück, das neben den Benutzerdaten auch den JWT enthält:

Dtos/AuthDtos.cs – LoginResponse

```
public record LoginResponse(int Id, string Email, string Name, string Token);
```

### 3.4 API-Endpunkte schützen ([Authorize])

Das `[Authorize]` -Attribut auf der Basisklasse `OAuthCrudController` schützt alle abgeleiteten Controller (`AdressenController`, `CitiesController`) automatisch. `AuthController` bleibt absichtlich ohne `[Authorize]` – Login und Registrierung müssen ohne Token erreichbar sein.

Controllers/ODataCrudController.cs – Basisklasse schützen

```
[Authorize] // Alle abgeleiteten Controller erfordern gültigen JWT
public abstract class ODataCrudController

```

Controllers/SettingsController.cs

```
[Authorize]
[Route("settings")]
[ApiController]
public class SettingsController : ControllerBase
{ /* ... */ }
```

### 3.5 Token im Frontend (auth.ts / api.ts)

NextAuth speichert den vom Backend erhaltenen JWT als `accessToken` in der verschlüsselten Session. Die neue Funktion `apiFetch()` liest den Token automatisch und setzt den `Authorization`-Header bei jedem API-Aufruf.

lib/auth.ts – Token aus Login-Response lesen und in Session speichern

```
// authorize(): Backend-Antwort enthält jetzt das Token-Feld
async authorize(credentials) {
    const res = await fetch(`${INTERNAL_API}/auth/login`, { ... });
    const user = await res.json(); // { id, email, name, token }
    if (!res.ok || !user) return null;
    return { id: String(user.id), email: user.email,
            name: user.name, accessToken: user.token };
},

// jwt-Callback: Token in JWT-Cookie weiterleiten
async jwt({ token, user }) {
    if (user) token.accessToken = user.accessToken;
    return token;
},

// session-Callback: Token an Frontend durchreichen
async session({ session, token }) {
    session.accessToken = token.accessToken as string;
    return session;
},
```

lib/api.ts – apiFetch() mit automatischem Authorization-Header

```

async function apiFetch(
  url: string,
  init: RequestInit = {}
): Promise<Response> {
  const session = await getSession();
  const token = session?.accessToken;

  const headers: Record<string, string> = {
    ...(init.headers as Record<string, string> | undefined ?? {}),
    'Content-Type': 'application/json',
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  };

  return fetch(url, { ...init, headers });
}

// Alle API-Funktionen nutzen apiFetch statt fetch:
export async function getAllAdressen(): Promise<Adresse[]> {
  const res = await apiFetch(ODATA_URL);
  if (!res.ok) throw new Error(`Fehler: ${res.statusText}`);
  const data = await res.json() as ODataResponse<Record<string, unknown>>;
  return data.value.map(normalizeAdresse);
}

```

## 4

## Architektur nach der Änderung

### Ablauf: Login und Token-Ausstellung



### Ablauf: Geschützter API-Aufruf



**200 OK + Daten** bei gültigem Token

**401 Unauthorized** bei ungültigem oder fehlendem Token

## JWT-Inhalt (Claims)

| Claim | Wert                     | Bedeutung                    |
|-------|--------------------------|------------------------------|
| sub   | z.B. 1                   | User-ID (Subject)            |
| email | admin@example.com        | E-Mail-Adresse               |
| name  | Admin                    | Anzeigename                  |
| jti   | GUID                     | Eindeutige Token-ID (JWT ID) |
| exp   | Unix-Timestamp           | Ablaufzeitpunkt (8 Stunden)  |
| iss   | AdressverwaltungApi      | Aussteller (Issuer)          |
| aud   | AdressverwaltungFrontend | Empfänger (Audience)         |

### Kein Passwort-Hash im Token

Der JWT enthält ausschliesslich die aufgeführten Claims. Sensitive Daten (Passwort-Hash, vollständige Adressdaten) werden niemals in den Token aufgenommen.

5

## Sicherheitshinweise

| Massnahme                   | Begründung                                                                                                                                   |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Schlüssel min. 32 Zeichen   | HMAC-SHA256 benötigt einen 256-Bit-Schlüssel. Kürzere Schlüssel werden von <code>AddJwtBearer</code> abgelehnt.                              |
| Schlüssel niemals in Git    | Umgebungsvariable <code>Jwt__Key</code> in <code>docker-compose.yml</code> verwenden; <code>appsettings.json</code> enthält nur Platzhalter. |
| Token-Gültigkeit: 8 Stunden | Entspricht einer Arbeitsschicht. Kurze Gültigkeit begrenzt den Schaden bei Token-Diebstahl.                                                  |
| Token nur in Session-Cookie | NextAuth speichert den Token im verschlüsselten HttpOnly-Cookie – nicht im <code>localStorage</code> (XSS-sicher).                           |

|                                                           |                                                                                                                 |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>AuthController</code> ohne <code>[Authorize]</code> | Login- und Register-Endpunkte müssen ohne Token erreichbar sein, damit sich Benutzer überhaupt anmelden können. |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|

|                                |                                                                 |
|--------------------------------|-----------------------------------------------------------------|
| Issuer und Audience validieren | Verhindert, dass Tokens aus anderen Diensten akzeptiert werden. |
|--------------------------------|-----------------------------------------------------------------|

### Nach Token-Ablauf: erneut anmelden

Nach 8 Stunden ist der Token abgelaufen. NextAuth leitet den Benutzer automatisch zur Login-Seite um. Es gibt keinen automatischen Token-Refresh in dieser Version – das ist für ein Schulprojekt bewusst vereinfacht.

6

## Migrationspfad (Upgrade von v2.1.0 auf v2.2.0)

### 1. Schlüssel generieren

Einen sicheren Zufallsschlüssel (min. 32 Zeichen) erstellen, z.B.:

```
openssl rand -base64 48
```

### 2. `docker-compose.yml` ergänzen

Umgebungsvariable `Jwt__Key` im Backend-Service setzen.

### 3. Docker-Container neu bauen

```
terminal
```

```
docker compose up --build -d
```

### 4. Kein Datenbank-Schema-Update nötig

Es sind keine EF-Core-Migrationen erforderlich. Bestehende Daten bleiben unverändert.

### 5. Benutzer müssen sich einmalig neu anmelden

Bestehende NextAuth-Sessions enthalten kein `accessToken`. Nach dem ersten Neustart müssen alle Benutzer das Session-Cookie löschen oder sich neu einloggen.

### Browser-Cookies löschen (falls Login-Probleme)

Chrome/Firefox: **F12** → **Application** → **Cookies** → **localhost:3000** → **alle löschen**, dann Seite neu laden und anmelden.

7

## Versionsverlauf

| Version | Datum | Beschreibung |
|---------|-------|--------------|
|---------|-------|--------------|

|               |                |                                                                                                                                                                                         |
|---------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v1.0.0        | 2024           | Initiale Version: Adressverwaltung mit OData, PostgreSQL, Next.js-Frontend                                                                                                              |
| v2.0.0        | 2024           | NextAuth.js, Google OAuth, PLZ-Wertehilfe (City-Entität), E-Mail-Benachrichtigung (SMTP)                                                                                                |
| v2.1.0        | 2026-09        | 14 Clean-Code-Befunde umgesetzt (SRP, DRY, Fehlerbehandlung, Benennung); neues Tutorial-Kapitel 14                                                                                      |
| <b>v2.2.0</b> | <b>2026-09</b> | <b>JWT-Authentifizierung im Backend: alle OData- und REST-Endpunkte mit Bearer-Token gesichert; Token-Ausstellung im AuthController; automatischer Authorization-Header im Frontend</b> |